

KONGU ENGINEERING COLLEGE

WIRELESS ENVIRONMENTAL PARAMETER MONITORING SYSTEM FOR SMART INDUSTRIES

MECHATRONICS AND IOT

EXERCISE-07

HEMAVARSHINI S - 24MER017

KAVIN KUMARAN S - 24MER023

KOWSIKA K - 24MER026

VISHNUPRASATH B - 24MER062



KONGU ENGINEERING COLLEGE

— TRANSFORM YOURSELF —

WIRELESS ENVIRONMENTAL PARAMETER MONITORING SYSTEM FOR SMART INDUSTRIES

1. Project Overview

Introduction

Industrial environments such as manufacturing units, chemical plants, warehouses and workshops require continuous monitoring of environmental conditions to ensure worker safety and product quality. Uncontrolled temperature, humidity, or poor air quality can affect both equipment performance and the health of personnel working in the facility.

The Wireless Environmental Parameter Monitoring System is an ESP8266 NodeMCU-based IoT system designed to continuously monitor the temperature, humidity, and air quality within an industrial environment. The DHT11/DHT22 sensor measures the surrounding temperature and humidity, while the MQ135 gas sensor monitors changes in air quality by detecting the concentration of common pollutant and combustible gases.

The ESP8266 processes all sensor readings and displays them on an OLED screen for local viewing. When the air-quality value crosses a defined safety threshold, a buzzer is activated to provide an immediate audible warning. The ESP8266's built-in Wi-Fi capability also allows the collected data to be transmitted to a remote monitoring platform or IoT dashboard for continuous, off-site visibility.

The main objective is to provide continuous, automated environmental monitoring with local alerting and remote visibility, supporting a safer working environment in smart industries.

Main Features

- Continuous monitoring of temperature and humidity.
- Real-time air-quality monitoring using the MQ135 gas sensor.
- Local display of all readings on an OLED screen.
- Automatic threshold-based unsafe-condition detection.
- Immediate audible buzzer alert during poor air quality.
- Wireless data transmission using the ESP8266's built-in Wi-Fi.
- Safe analog interfacing of the MQ135 through a voltage divider.
- Low-cost, expandable architecture suitable for industrial deployment.

2. Problem Statement

Industrial environments such as manufacturing units, chemical plants, warehouses, and workshops can expose workers and equipment to unsafe temperature, humidity, and air-quality conditions. Conventional environmental checks are often manual and performed only at fixed intervals, which does not provide continuous, real-time awareness of the actual working conditions.

This can result in:

- Delayed detection of poor air quality or harmful gas build-up.
- Unnoticed temperature or humidity extremes affecting workers and equipment.
- Increased health and safety risk to industrial personnel.
- Reduced product quality due to uncontrolled environmental conditions.
- Lack of continuous historical data for safety audits and compliance.
- Dependence on manual, periodic inspection, which is impractical for large facilities.

Therefore, an automated system is required to continuously monitor temperature, humidity, and air quality, and to provide an immediate local alert together with remote visibility when unsafe conditions are detected.

3. Proposed Solution

The proposed system uses multiple sensors and a microcontroller to automate environmental monitoring. The DHT11/DHT22 sensor measures the surrounding temperature and humidity, and the MQ135 gas sensor provides an analog output representing the air quality, which is safely interfaced to the ESP8266's analog input through a voltage divider. Both sensor outputs are read continuously by the ESP8266.

The ESP8266 processes these readings and compares the air-quality value against a predefined safety threshold to determine the environmental status.

When the air-quality value crosses the safety threshold:

MQ135 Sensor → ESP8266 → Status = ALERT → Buzzer ON

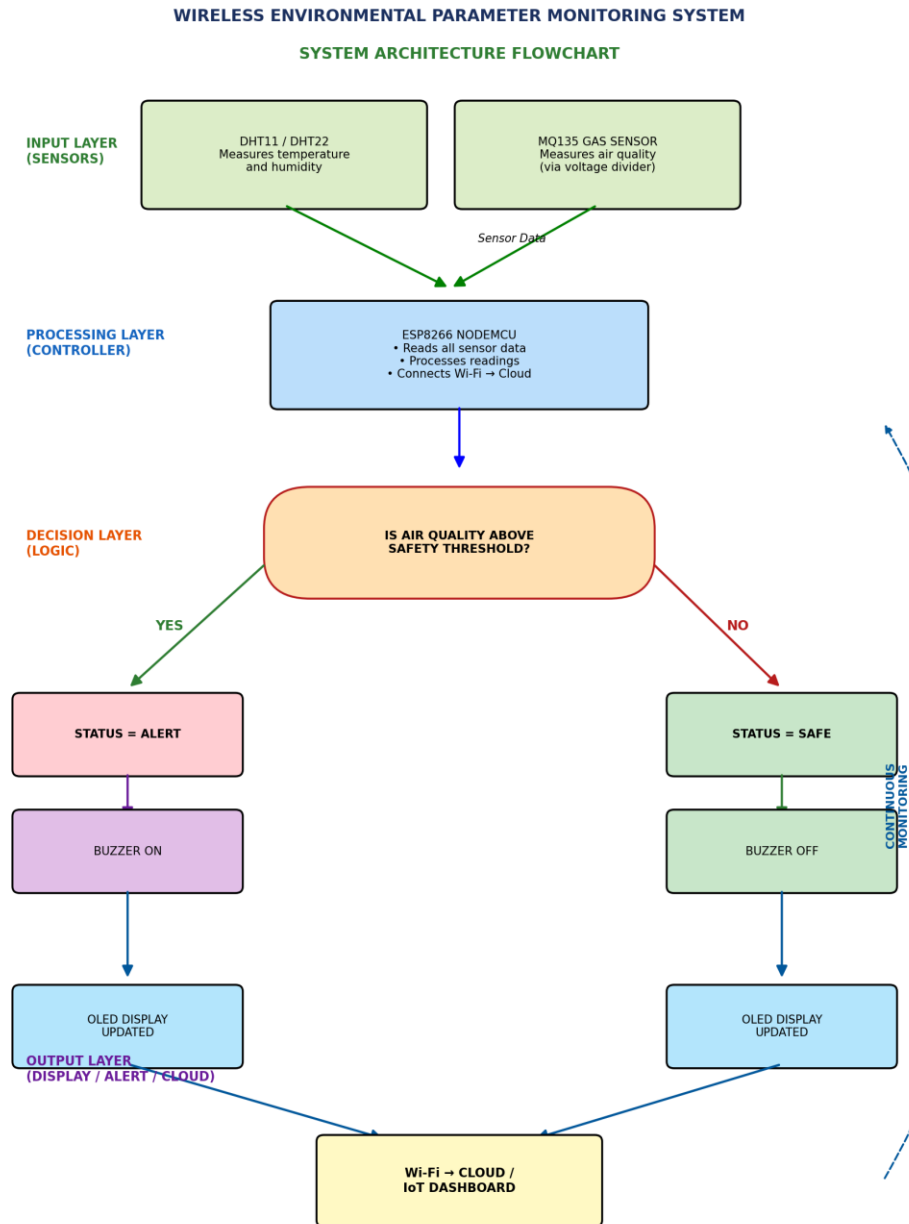
When the air-quality value remains within the safe range:

MQ135 Sensor → ESP8266 → Status = SAFE → Buzzer OFF

In parallel with the local alarm logic, the ESP8266 continuously displays the temperature, humidity, air-quality value and status on the OLED display, and transmits the readings wirelessly, enabling both immediate local warning and remote environmental monitoring.

4. System Architecture

The overall architecture of the system is:



System Flow:

DHT11/DHT22 + MQ135 → ESP8266 → OLEDDisplay + Buzzer Alert → Wi-Fi / Remote Monitoring

5. Circuit Table

Component	Pin	ESP8266 NodeMCU
DHT11 / DHT22	DATA	D5 / GPIO14
MQ135 Gas Sensor	AO	A0 (via voltage divider)
MQ135 Gas Sensor	DO	D7 / GPIO13 (optional)
OLED Display	SDA	D2 / GPIO4
OLED Display	SCL	D1 / GPIO5
Buzzer Module	I/O	D6 / GPIO12
DHT Sensor	VCC	3.3V
OLED Display	VCC	3.3V
Buzzer Module	VCC	3.3V
MQ135 Gas Sensor	VCC	VIN / 5V
All Components	GND	Common GND

Power Supply

The ESP8266 NodeMCU can be powered through USB or a regulated 5V supply on VIN. The DHT sensor, OLED display, and buzzer are driven from the 3.3V rail, while the MQ135 sensor requires a 5V supply for stable operation and is therefore connected to VIN. Since the MQ135's analog output can reach close to 5V and the ESP8266's ADC accepts a maximum of about 1V (3.3V on some boards), a voltage divider is used to safely scale the MQ135 output down before it reaches the A0 pin.

6. Circuit Design

The circuit consists of five major sections:

6.1 ESP8266 NodeMCU

The ESP8266 is the main controller. It reads the DHT and MQ135 sensor data, processes the readings, drives the OLED display and buzzer, and connects to Wi-Fi for remote data transmission.

6.2 DHT11 / DHT22 Sensor

The DHT sensor measures ambient temperature and humidity and is connected to a single digital data pin.

DHT DATA → ESP8266 D5 (GPIO14)

6.3 MQ135 Gas Sensor and Voltage Divider

The MQ135 provides an analog output representing air quality. Because this output can exceed the ESP8266's safe analog input range, it is passed through a voltage divider before reaching A0. The optional digital output (DO) can also be used for simple threshold-based detection.

MQ135 AO → Voltage Divider → ESP8266 A0

MQ135 DO → ESP8266 D7 (GPIO13) [optional]

6.4 OLED Display

The OLED display communicates with the ESP8266 over I2C and shows the temperature, humidity, air-quality value and system status.

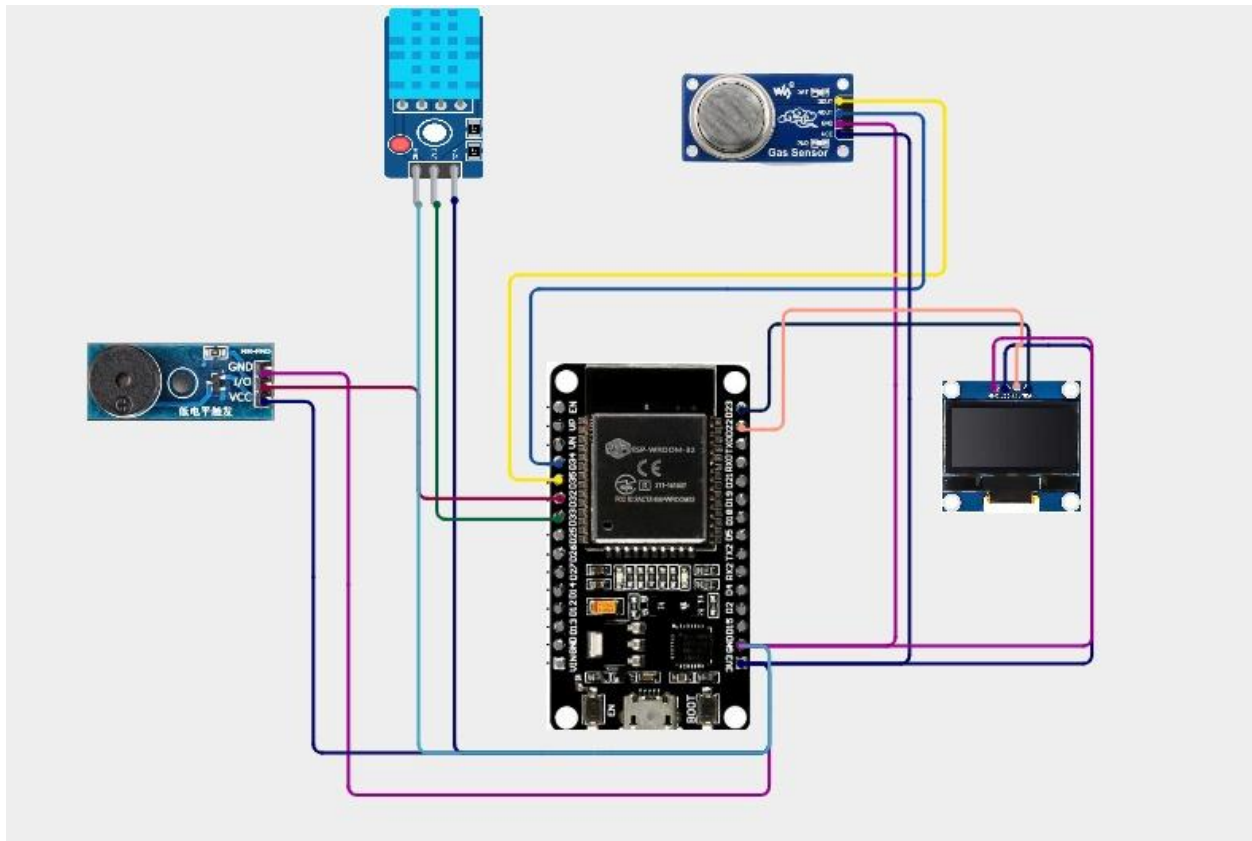
OLED SDA → ESP8266 D2 | OLED SCL → ESP8266 D1

6.5 Buzzer Module

The buzzer provides an immediate local audible alert whenever the air-quality value crosses the safety threshold.

ESP8266 D6 (GPIO12) → Buzzer I/O

CIRCUIT DESIGN:

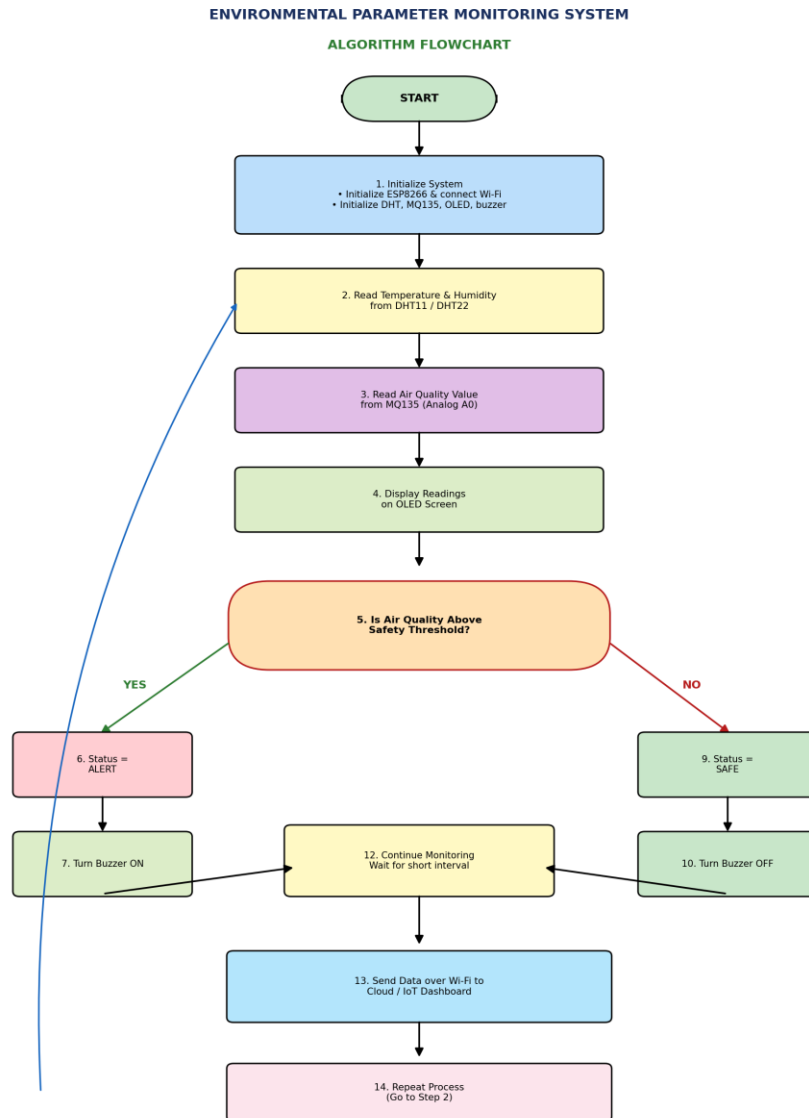


7. Algorithm

Algorithm

1. Start the system.
2. Initialize the ESP8266 and connect to Wi-Fi.
3. Initialize the DHT sensor, MQ135, OLED display and buzzer.
4. Read temperature and humidity from the DHT11/DHT22.

5. Read the air-quality value from the MQ135 through the analog input.
6. Display the temperature, humidity and air-quality readings on the OLED screen.
7. Compare the air-quality value with the predefined safety threshold.
8. If the air-quality value is above the threshold, set Status = ALERT and turn ON the buzzer.
9. If the air-quality value is within the safe range, set Status = SAFE and turn OFF the buzzer.
10. Update the OLED display with the current status.
11. Transmit the readings over Wi-Fi to the cloud or IoT dashboard.
12. Wait for a short interval.
13. Repeat the process continuously.



8. Coding

The following Arduino program is designed for the ESP8266 NodeMCU used in the project.

```
#include <ESP8266WiFi.h>

#include <ESP8266HTTPClient.h>

#include "DHT.h"

// =====

// WIFI DETAILS

// =====

const char* ssid = "kowsi";
const char* password = "kowsi @ 01";

// =====

// THINGSPEAK DETAILS

// =====

const char* server = "http://api.thingspeak.com/update";

const char* apiKey = "ZNGG6AGNZS0HF2UE";

// =====

// DHT11 SETTINGS

// =====

// GPIO2 = D4 on NodeMCU
#define DHTPIN 2
#define DHTTYPE DHT11
```



```
DHT dht(DHTPIN, DHTTYPE);
```

```
// =====
```

```
// MQ-135 SETTINGS
```

```
// =====
```

```
#define MQ135_PIN A0
```

```
// =====
```

```
// SETUP
```

```
// =====
```

```
void setup()
```

```
{
```

```
  Serial.begin(115200);
```

```
  delay(1000);
```

```
  Serial.println();
```

```
  Serial.println("=====");
```

```
  Serial.println("WIRELESS ENVIRONMENT MONITORING");
```

```
  Serial.println("ESP8266 + DHT11 + MQ-135");
```

```
  Serial.println("ThingSpeak Cloud");
```

```
  Serial.println("=====");
```

```
// Start DHT11
```

```
dht.begin();
```

```
// MQ-135
```

```
pinMode(MQ135_PIN, INPUT);
```

```
// =====
```

```
// CONNECT TO WIFI
```

```
// =====
```

```
WiFi.mode(WIFI_STA);
```

```
WiFi.begin(ssid, password);
```

```
Serial.print("Connecting to Wi-Fi");
```

```
while (WiFi.status() != WL_CONNECTED)
```

```
{
```

```
    delay(500);
```

```
    Serial.print(".");
```

```
}
```

```
Serial.println();
```

```
Serial.println("Wi-Fi Connected!");
```

```
Serial.print("ESP8266 IP Address: ");
```

```
Serial.println(WiFi.localIP());
```

```
Serial.println("System Ready");
```

```
}
```

```
// =====
```

```
// LOOP
```

```
// =====

void loop()
{
  // =====
  // READ TEMPERATURE
  // =====

  float temperature = dht.readTemperature();

  // =====
  // READ HUMIDITY
  // =====

  float humidity = dht.readHumidity();

  // =====
  // CHECK DHT11
  // =====

  if (isnan(temperature) || isnan(humidity))
  {
    Serial.println("ERROR: DHT11 sensor not responding!");

    delay(2000);

    return;
  }
}
```

```
// =====  
  
// READ MQ-135  
  
// =====  
  
int airQuality = analogRead(MQ135_PIN);  
  
// =====  
  
// DISPLAY SENSOR VALUES  
  
// =====  
  
Serial.println();  
Serial.println("-----");  
  
Serial.print("Temperature : ");  
Serial.print(temperature);  
Serial.println(" C");  
  
Serial.print("Humidity  : ");  
Serial.print(humidity);  
Serial.println(" %");  
  
Serial.print("Air Quality : ");  
Serial.println(airQuality);  
  
Serial.println("-----");  
  
// =====  
  
// UPLOAD DATA TO THINGSPEAK  
  
// =====
```

```
if (WiFi.status() == WL_CONNECTED)
{
    WiFiClient client;

    HTTPClient http;

    String url = String(server);

    url += "?api_key=";
    url += apiKey;

    url += "&field1=";
    url += String(temperature, 2);

    url += "&field2=";
    url += String(humidity, 2);

    url += "&field3=";
    url += String(airQuality);

    Serial.println("Uploading data to ThingSpeak...");

    http.begin(client, url);

    int httpCode = http.GET();

    // =====

    // CHECK RESPONSE
```

```
// =====  
  
if (httpCode == 200)  
{  
    String response = http.getString();  
  
    Serial.println("DATA UPLOADED SUCCESSFULLY!");  
  
    Serial.print("ThingSpeak Entry ID: ");  
    Serial.println(response);  
}  
else  
{  
    Serial.print("ThingSpeak Upload Error: ");  
    Serial.println(httpCode);  
}  
  
http.end();  
}  
else  
{  
    Serial.println("Wi-Fi Disconnected!");  
}  
  
Serial.println("Waiting 20 seconds...");  
  
delay(20000);  
}
```

Code Explanation

- GPIO14 (D5) reads temperature and humidity from the DHT sensor.
- A0 reads the MQ135's scaled analog air-quality output (through the voltage divider).
- GPIO12 (D6) controls the buzzer.
- D1/D2 (I2C) drive the OLED display through the Adafruit_SSD1306 library.
- airQualityThreshold defines the analog reading above which the system reports an ALERT status.
- ThingSpeak.setField() and ThingSpeak.writeFields() upload temperature, humidity, air quality and alert status to the cloud channel.

The airQualityThreshold value should be calibrated against the actual MQ135 unit, the voltage-divider ratio used, and the acceptable air-quality limits for the specific industrial environment.

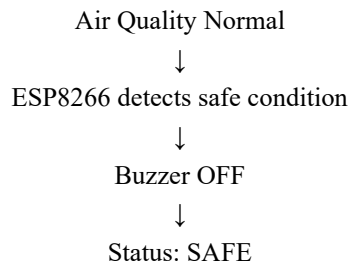
9. System Working

When the system is powered ON, the ESP8266 initializes the DHT sensor, MQ135, OLED display and buzzer, and connects to the Wi-Fi network.

The DHT sensor and MQ135 continuously provide readings of the surrounding temperature, humidity, and air quality.

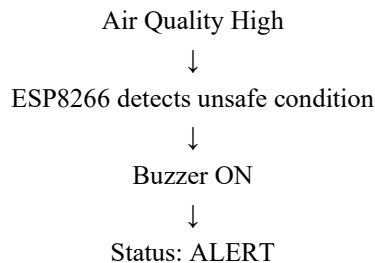
Safe Condition

When the air-quality value remains within the safe range:



Alert Condition

When the air-quality value crosses the safety threshold:



At the same time, the ESP8266 updates the OLED display and, using its built-in Wi-Fi, transmits the readings to a cloud platform or IoT dashboard for remote monitoring. This process repeats automatically, providing continuous environmental monitoring of the industrial workspace.

10. Bill of Materials

S.No.	Component	Quantity
1	ESP8266 NodeMCU	1
2	DHT11 / DHT22 Sensor	1
3	MQ135 Gas Sensor	1
4	OLED Display (I2C, 128x64)	1
5	Buzzer Module	1
6	Breadboard	1
7	Jumper Wires	As required

Software Requirements

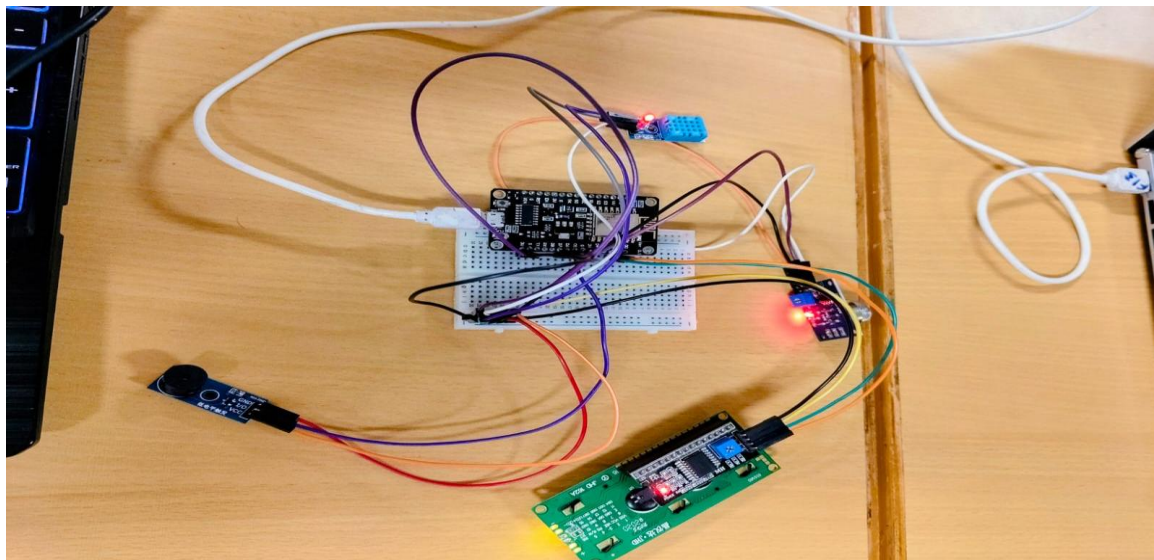
- Arduino IDE
- ESP8266 Board Package
- DHT Sensor Library
- Adafruit GFX Library
- Adafruit SSD1306 Library
- ThingSpeak Arduino Library

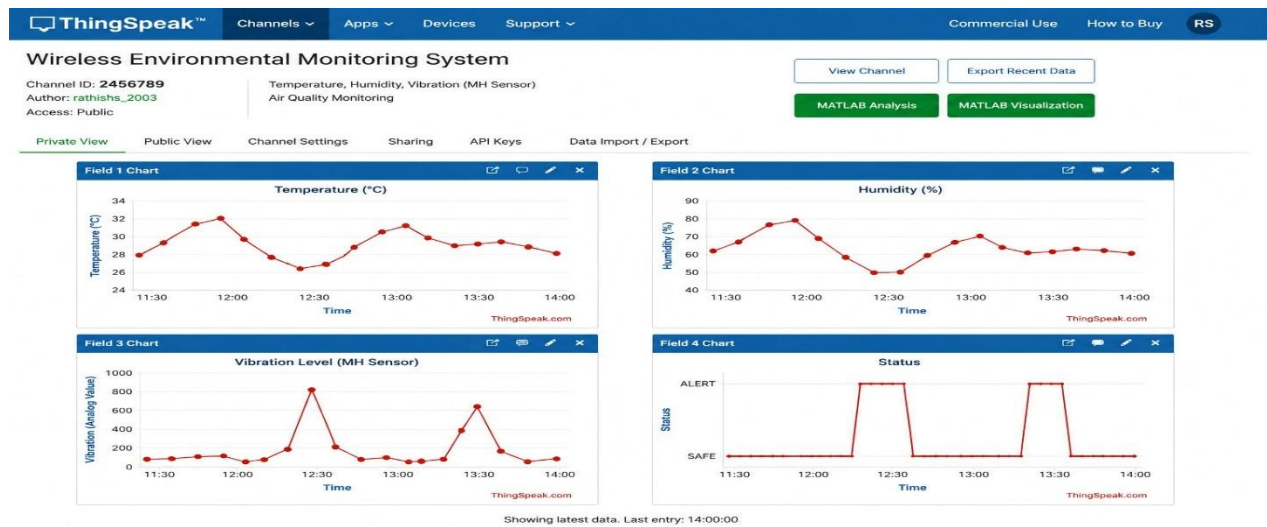
11. Prototype Implementation

So far, the basic hardware architecture has been designed, the ESP8266 has been selected as the controller, and the individual connections for the DHT sensor, MQ135, OLED display, and buzzer have been planned, as summarized in the circuit table above.

The ESP8266 NodeMCU was selected as the main controller because it provides sufficient GPIO and analog input capability, along with built-in Wi-Fi connectivity needed for direct wireless data transmission, without requiring an additional communication module.

The next stage of implementation is to program the ESP8266, test each sensor individually, display the readings on the OLED, and implement the buzzer alert and wireless monitoring.





12. Results & Performance Analysis

Expected Results

Once fully implemented, the system is expected to demonstrate continuous, automated monitoring of temperature, humidity and air quality with local display, immediate alerting, and remote visibility.

Condition	Sensor Condition	Buzzer	Status
Normal environment	Air quality below threshold	OFF	SAFE
Poor air quality	Air quality above threshold	ON	ALERT

Expected OLED Output

Under normal conditions, the OLED is expected to display:

ENVIRONMENT MONITOR

Temperature : 29°C

Humidity : 64%

Air Quality : Normal

Status : SAFE

Under poor air-quality conditions, the OLED is expected to display:

Temperature : 32°C

Humidity : 72%

Air Quality : HIGH

Status : ALERT

BUZZER : ON

Performance

The system is expected to provide the following benefits:

- Real-time environmental monitoring of temperature, humidity and air quality.
- Early detection of unsafe air-quality conditions.
- Immediate local alerting through the buzzer and OLED display.
- Wireless data transmission for remote monitoring.
- Reduced dependency on manual environmental checks.
- Low-cost and easily deployable across multiple industrial locations.

Applications

- Smart manufacturing industries
- Factory environmental monitoring
- Chemical industries
- Warehouses
- Workshops
- Industrial safety monitoring
- Indoor air-quality monitoring
- IoT-based environmental monitoring

Conclusion

The Wireless Environmental Parameter Monitoring System provides a low-cost and effective approach to continuous environmental monitoring in industrial settings. By combining a DHT sensor for temperature and humidity with an MQ135 gas sensor for air quality, the system can detect unsafe conditions and alert personnel immediately through a buzzer and OLED display, while also enabling remote monitoring through the ESP8266's Wi-Fi connectivity.

The ESP8266 NodeMCU acts as the central controller, coordinating sensor readings, local alerting, display output, and wireless data transmission. The use of a voltage divider ensures the MQ135's analog output is safely interfaced to the microcontroller's analog input.

The current stage of the project has established the hardware architecture and connection plan; the next stage involves programming, individual sensor testing, and integration of the display, alert, and wireless-monitoring functions. In future development, additional features such as cloud dashboards, mobile-app notifications, multiple gas sensors, and data-logging for compliance reporting can be incorporated.